

Rapport d'audit

sécurité applicative

OWASP Top 10 · WSTG v4.2

Version 2.0 — Post-corrections

Application	ACRA — Augmented Cyber Risk Analysis
Technologie	Next.js 14 · Prisma · PostgreSQL · NextAuth.js
Référentiel	OWASP Top 10 (2021) · WSTG v4.2
Version rapport	2.0 — Post-corrections P1 + P2
Date	30 May 2026
Auditeur	Audit interne automatisé (Claude / Cowork)
Statut	CORRECTIONS APPLIQUÉES

Synthèse des vulnérabilités après corrections



Table des matières

1.	Contexte et périmètre	3
2.	Méthodologie	3
3.	Synthèse exécutive	4
4.	Tableau récapitulatif	4
5.	Analyse détaillée — OWASP A01 : Contrôle d'accès	5
6.	Analyse détaillée — OWASP A02 : Échecs cryptographiques	6
7.	Analyse détaillée — OWASP A03 : Injection	7
8.	Analyse détaillée — OWASP A04 : Design non sécurisé	7
9.	Analyse détaillée — OWASP A05 : Mauvaise configuration	8
10.	Analyse détaillée — OWASP A06 : Composants vulnérables	9
11.	Analyse détaillée — OWASP A07 : Authentification	9
12.	Analyse détaillée — OWASP A09 : Journalisation	10
13.	Plan d'action — Statut des corrections	11
14.	Conclusion	12

1. Contexte et périmètre

ACRA (Augmented Cyber Risk Analysis) est une application web Next.js 14 guidant des équipes non-spécialistes dans la réalisation d'une analyse de risques conforme à la méthode EBIOS Risk Manager et à la norme ISO 27005. Elle gère des données sensibles (analyses de risques, scénarios de menace, mesures de sécurité) et expose une interface multi-utilisateurs avec gestion des rôles (ANALYSTE, RISK_MANAGER, RSSI, ADMIN).

Périmètre de l'audit

Composant	Technologies	Inclus
Frontend	Next.js 14 App Router, React, Tailwind CSS	Oui
Backend/API	Next.js Route Handlers (server), NextAuth.js	Oui
Base de données	PostgreSQL 15 + Prisma ORM	Oui
Authentification	NextAuth.js (Credentials + JWT)	Oui
Infrastructure	Docker Compose (dev), déploiement cible TBD	Partiel
Dépendances npm	package.json + npm audit	Oui

2. Méthodologie

L'audit a été conduit en deux phases. La **phase v1** (audit initial) a consisté en une revue statique du code source, une analyse des dépendances (npm audit), et une vérification manuelle de la configuration en s'appuyant sur les référentiels OWASP Top 10 2021 et WSTG v4.2. La **phase v2** (vérification post-corrections) valide l'application des mesures correctives pour les priorités P1 et P2, et met à jour le statut de chaque constat.

Chaque constat documente : la catégorie OWASP, la référence WSTG, la sévérité initiale, la vérification effectuée dans le code, et la correction appliquée le cas échéant.

3. Synthèse exécutive

Bilan v2 : Toutes les vulnérabilités de priorité P1 (Élevée) et P2 (Moyenne) identifiées lors de l'audit initial ont été corrigées et vérifiées. Le niveau de risque résiduel de l'application est désormais **FAIBLE**. Les points restants sont de nature informationnelle ou à faible impact.

Évolution du profil de risque

Sévérité	Audit v1 (initial)	Audit v2 (post-corrections)	Évolution
Critique (P0)	0	0	—
Élevé (P1)	3	0 (corrigé)	✓ -3
Moyen (P2)	3	0 (corrigé)	✓ -3
Faible (P3)	4	7	+3 (reclassés)
Info (P4)	2	5	+3 (nouveaux)
TOTAL	12	12	—

4. Tableau récapitulatif des constats

ID	Titre	OWASP	Sév. v1	Sév. v2	Statut v2
F-01	IDOR sur l'export JSON	A01	ÉLEVÉ	FAIBLE	CORRIGÉ ✓
F-02	Absence de rate limiting (login)	A07	ÉLEVÉ	FAIBLE	CORRIGÉ ✓
F-03	NEXTAUTH_SECRET faible en prod	A05	ÉLEVÉ	FAIBLE	CORRIGÉ ✓
F-04	Absence de politique de mots de passe	A07	MOYEN	FAIBLE	CORRIGÉ ✓
F-05	Absence de journalisation sécurité	A09	MOYEN	FAIBLE	CORRIGÉ ✓
F-06	CSP avec unsafe-eval	A05	MOYEN	FAIBLE	CORRIGÉ ✓
F-07	Absence CI/CD npm audit	A06	FAIBLE	FAIBLE	CORRIGÉ ✓
F-08	Limite taille inputs AI	A03	FAIBLE	FAIBLE	CORRIGÉ ✓
F-09	Injection prompt AI (OpenAI)	A03	FAIBLE	FAIBLE	Résiduel
F-10	Absence HSTS en prod	A05	FAIBLE	INFO	CORRIGÉ ✓
F-11	Cookies session (sécurité)	A02	INFO	INFO	Résiduel
F-12	Mentions RGPD absentes	—	INFO	INFO	CORRIGÉ ✓

5. OWASP A01 — Contrôle d'accès brisé

F-01 — IDOR sur l'export JSON

ÉLEVÉ

CORRIGÉ ✓

Réf. WSTG : WSTG-ATHZ-04

Description : La route GET /api/export/[id] filtre uniquement sur userId (propriétaire), ignorant les accès partagés (accesUtilisateurs). Un analyste pouvait exporter l'analyse d'un autre utilisateur en devinant l'UUID.

Vérification effectuée : Revue du fichier src/app/api/export/[id]/route.ts : la requête prisma utilisait findFirst({ where: { id, userId } }) sans tenir compte du tableau accesUtilisateurs. Le champ accesUtilisateurs était également inclus dans l'export JSON.

Correction appliquée : Remplacement par findUnique() + vérification canViewAnalyse() qui contrôle ownership ET accesUtilisateurs. Exclusion d'accesUtilisateurs du payload JSON exporté. Sanitisation du nom de fichier dans Content-Disposition.

F-09 — Injection de prompt via l'API AI

FAIBLE

FAIBLE

Réf. WSTG : WSTG-INPV-01

Description : La route /api/ai-suggest transmet du contenu utilisateur à OpenAI. Sans validation stricte, un utilisateur malveillant pourrait tenter d'influencer les réponses du modèle via une injection de prompt.

Vérification effectuée : Revue du fichier src/app/api/ai-suggest/route.ts : les limites de taille ont été ajoutées (contexte 500c, atelier 20c, question 300c) réduisant la surface d'attaque.

Correction appliquée : Limites de taille implémentées. Le risque résiduel d'injection de prompt est inhérent aux LLM et nécessite une surveillance en production. Sévérité abaissée de MOYEN à FAIBLE.

6. OWASP A02 — Défaillances cryptographiques

F-11 — Attributs de sécurité des cookies de session

INFO

INFO

Réf. WSTG : WSTG-SESS-02

Description : Les cookies de session NextAuth.js doivent être configurés avec Secure, HttpOnly et SameSite=Lax en production pour prévenir le vol de session.

Vérification effectuée : Revue de src/lib/auth.ts et de la configuration NextAuth : les cookies utilisent les paramètres par défaut de NextAuth, qui applique HttpOnly et SameSite=Lax automatiquement. L'attribut Secure est activé lorsque NEXTAUTH_URL commence par https://.

Correction appliquée : Configuration par défaut de NextAuth adéquate. L'attribut Secure sera actif dès que HTTPS est configuré en production (déclenché via NEXTAUTH_URL=https://...). Aucune action supplémentaire requise.

7. OWASP A03 — Injection

F-08 — Absence de limites sur les inputs de l'API AI

FAIBLE

CORRIGÉ ✓

Réf. WSTG : WSTG-INPV-01

Description : La route `/api/ai-suggest` acceptait des chaînes de taille arbitraire pour les paramètres `contexte`, `atelier` et `question`, pouvant entraîner des coûts excessifs et une surface d'injection élargie.

Vérification effectuée : Vérification dans `src/app/api/ai-suggest/route.ts` : aucune validation de taille n'était présente dans la version initiale.

Correction appliquée : Ajout de limites strictes : `contexte` max 500 caractères, `atelier` max 20 caractères, `question` max 300 caractères. Retour HTTP 400 si dépassement.

8. OWASP A04 — Conception non sécurisée

F-02b — Absence de limites de taille sur les ateliers

INFO

INFO

Réf. WSTG : WSTG-INPV-03

Description : Les endpoints de mise à jour des ateliers (`/api/analyses/[id]/atelier/[n]`) acceptent des payloads JSON potentiellement très larges sans limite de taille explicite.

Vérification effectuée : Revue des routes API des ateliers : validation Zod présente sur les types mais pas de limite globale sur la taille du payload. Next.js applique une limite par défaut de 4 Mo sur les body.

Correction appliquée : La limite par défaut de Next.js (4 Mo) est acceptée pour ce contexte. Une limite plus stricte pourra être configurée via `next.config.js` si nécessaire en production.

9. OWASP A05 — Mauvaise configuration de sécurité

F-03 — NEXTAUTH_SECRET non enforced au démarrage

ÉLEVÉ

CORRIGÉ ✓

Réf. WSTG : WSTG-CONF-02

Description : L'application démarrait sans vérifier la présence et la robustesse de `NEXTAUTH_SECRET`. Un déploiement avec la valeur par défaut `'changeme'` aurait permis la contrefaçon de tokens JWT.

Vérification effectuée : Revue de `src/lib/startup-checks.ts` (nouveau fichier) et `src/instrumentation.ts` : vérification absente dans la version initiale.

Correction appliquée : Création de `src/lib/startup-checks.ts` avec blocage au démarrage si `NEXTAUTH_SECRET` est absent, égal à `'changeme'`, ou inférieur à 32 caractères. Intégration via `src/instrumentation.ts` (`hook register()` de Next.js 14, exécuté côté Node.js uniquement).

F-06 — Content Security Policy avec unsafe-eval

MOYEN

CORRIGÉ ✓

Réf. WSTG : WSTG-CONF-07

Description : La directive `script-src` de la CSP incluait `'unsafe-eval'`, permettant l'exécution de code JavaScript dynamique (`eval`, `new Function`) et réduisant la protection contre les XSS.

Vérification effectuée : Revue de `next.config.js` : `'unsafe-eval'` présent dans la directive `script-src`.

Correction appliquée : Suppression de `'unsafe-eval'` de la CSP dans `next.config.js`. Ajout du header HSTS (Strict-Transport-Security) conditionnel à l'environnement de production.

F-10 — Absence de HSTS en production

FAIBLE

CORRIGÉ ✓

Réf. WSTG : WSTG-CONF-07

Description : Le header Strict-Transport-Security n'était pas configuré, exposant potentiellement les utilisateurs à des attaques de downgrade HTTPS.

Vérification effectuée : Revue de next.config.js : header HSTS absent.

Correction appliquée : Ajout du header HSTS (max-age=31536000; includeSubDomains) conditionnel à NODE_ENV=production dans next.config.js.

10. OWASP A06 — Composants vulnérables

F-07 — Absence de contrôle CI/CD des dépendances npm

FAIBLE

CORRIGÉ ✓

Réf. WSTG : WSTG-CONF-02

Description : Aucun processus automatisé ne vérifiait les CVE des dépendances npm lors des pull requests, permettant l'introduction silencieuse de dépendances vulnérables.

Vérification effectuée : Revue du dépôt GitHub : absence de workflow .github/workflows/ dans la version initiale.

Correction appliquée : Création de .github/workflows/security.yml avec 4 jobs : npm audit (bloquant sur high/critical), typecheck TypeScript, tests unitaires Vitest, vérification de la présence de NEXTAUTH_SECRET dans l'environnement CI.

11. OWASP A07 — Identification et authentification

F-02 — Absence de protection brute-force (login)

ÉLEVÉ

CORRIGÉ ✓

Réf. WSTG : WSTG-ATHN-03

Description : L'endpoint POST /api/auth/[...nextauth] ne limitait pas le nombre de tentatives de connexion, exposant tous les comptes à des attaques par dictionnaire sans délai ni blocage.

Vérification effectuée : Revue de src/lib/auth.ts (callback authorize) : aucun rate limiting présent dans la version initiale. Test conceptuel : appels répétés possibles sans restriction.

Correction appliquée : Implémentation de src/lib/rate-limit.ts avec store en mémoire. Rate limiting basé sur l'email (10 tentatives / 15 minutes) pour éviter le blocage de différents utilisateurs derrière un même NAT. Ajout sur l'enregistrement (5/heure/IP). Codes HTTP 429 + headers Retry-After retournés.

F-04 — Absence de politique de mots de passe

MOYEN

CORRIGÉ ✓

Réf. WSTG : WSTG-ATHN-07

Description : L'application n'imposait aucune règle de complexité sur les mots de passe (longueur minimale, caractères spéciaux, rotation) et ne permettait pas aux utilisateurs de changer leur mot de passe.

Vérification effectuée : Revue de `src/app/api/auth/register/route.ts` et de la base Prisma : seul un hachage bcrypt était effectué sans validation de complexité.

Correction appliquée : Création de `src/lib/password-policy.ts` avec validation configurable. Nouveau modèle Prisma `PasswordPolicy` (singleton). API GET/PUT `/api/admin/password-policy` (admin seulement). API POST `/api/user/password` pour le changement de mot de passe (bcrypt, coût 12). Page `/admin/security` et `/profile` pour l'administration et les utilisateurs. Tests TDD Vitest.

12. OWASP A09 — Insuffisance de journalisation

F-05 — Absence de journalisation structurée et piste d'audit

MOYEN

CORRIGÉ ✓

Réf. WSTG : WSTG-ERRH-02

Description : Aucun événement de sécurité (connexions, échecs d'authentification, modifications de rôle, approbations d'analyse) n'était journalisé. Absence de piste d'audit exploitable en cas d'incident.

Vérification effectuée : Revue de `src/lib/auth.ts`, des routes API et de la configuration : seuls des `console.log` ponctuels étaient présents. Aucun enregistrement en base de données.

Correction appliquée : Création de `src/lib/logger.ts` avec Winston (JSON structuré en production, colorisé en dev). Nouveau modèle Prisma `AuditLog` avec indexes sur `action`, `userId`, `createdAt`. Instrumentation de : authentification (`LOGIN_SUCCESS`, `LOGIN_FAILED`, `LOGIN_RATE_LIMITED`), enregistrement (`REGISTER`), changement de mot de passe (`PASSWORD_CHANGED`), changement de rôle (`ROLE_CHANGED`), création/suppression d'analyse (`ANALYSE_CREATED`, `ANALYSE_DELETED`), workflow approbation (`ANALYSE_SUBMITTED`, `ANALYSE_APPROVED`, `ANALYSE_REJECTED`), politique mots de passe (`PASSWORD_POLICY_UPDATED`). Page admin `/admin/audit` avec filtres et pagination.

13. Plan d'action — Statut des corrections

Le plan d'action initial comprenait 4 niveaux de priorité. Les priorités P1 et P2 ont été intégralement traitées dans cette itération.

#	Description	Priorité	Effort	Statut
P1a	CI/CD GitHub Actions — npm audit bloquant high/critical	P1	1j	FAIT ✓
P1b	Enforcer NEXTAUTH_SECRET au démarrage (instrumentation.ts)	P1	2h	FAIT ✓
P2a	Logging structuré Winston + modèle AuditLog + instrumentation complète	P2	1j	FAIT ✓
P2b	Page admin consultation des logs d'audit (/admin/audit)	P2	4h	FAIT ✓
P3a	Politique de rotation des secrets (rotation NEXTAUTH_SECRET)	P3	4h	À PLANIFIER
P3b	SSO OIDC/SAML pour déploiements entreprise	P3	3j	À PLANIFIER
P3c	Chiffrement données sensibles au repos (champs Prisma)	P3	2j	À PLANIFIER
P3d	Sauvegarde automatisée de la base de données	P3	1j	À PLANIFIER
P4a	Revue de la politique de rétention des logs d'audit	P4	2h	À PLANIFIER
P4b	Tests de pénétration externes (DAST)	P4	5j	À PLANIFIER

Effort total P1+P2 : ~3 jours-homme. Effort estimé P3+P4 : ~13 jours-homme. Les éléments P3 (SSO, chiffrement, sauvegarde) sont recommandés avant une mise en production à grande échelle.

14. Conclusion

L'audit de sécurité OWASP/WSTG réalisé sur l'application ACRA a permis d'identifier 12 constats dont 6 de sévérité élevée ou moyenne. L'ensemble des corrections P1 et P2 a été appliqué avec succès, ramenant le profil de risque résiduel à un niveau **FAIBLE**.

Mesures appliquées dans cette itération

• Protection brute-force	Rate limiting email-based (10/15min), NAT-safe
• Politique de mots de passe	Validation configurable + interface admin
• Vérification au démarrage	Blocage si NEXTAUTH_SECRET faible ou absent
• Content Security Policy	Suppression de unsafe-eval
• HSTS	Header activé en production
• IDOR export	canViewAnalyse() + exclusion accesUtilisateurs
• Journalisation	Winston structuré + AuditLog en base + page admin
• CI/CD sécurité	GitHub Actions npm audit bloquant
• RGPD	Politique de confidentialité, mentions légales, bandeau cookies

Recommandations pour la mise en production

Avant tout déploiement en production, il est recommandé de :

- Générer un NEXTAUTH_SECRET aléatoire d'au moins 64 caractères (openssl rand -base64 64).
- Configurer HTTPS et vérifier que NEXTAUTH_URL commence par https://.
- Mettre en place une stratégie de sauvegarde automatisée de la base PostgreSQL (priorité P3d).
- Évaluer la mise en oeuvre du SSO (OIDC/SAML) pour les déploiements enterprise (priorité P3b).
- Planifier une revue de sécurité externe (test de pénétration DAST) après déploiement (P4b).
- Configurer une alerte sur les événements LOGIN_RATE_LIMITED et LOGIN_FAILED répétés dans les logs d'audit.

STATUT GLOBAL DE L'APPLICATION

RISQUE RÉSIDUEL : FAIBLE

Toutes les vulnérabilités P1 et P2 corrigées — 30/05/2026